SQL

SQL Interview Pack

8 patterns behind most SQL interview questions. Learn the pattern once, solve 50 questions with it.

- **8 patterns with ready queries**
- **The JOIN mistake that doubles rows**
- **Order of execution, explained**

@thedataguy16

Patterns 1 to 4

Window functions. If you know these, you clear most product and service company rounds.

Top N per group

Top 3 salaries in each department. DENSE_RANK keeps ties.

```
DENSE_RANK() OVER (PARTITION BY dept
  ORDER BY salary DESC) AS rk  -- then
WHERE rk <= 3
```

Running total

Cumulative revenue by date.

```
SUM(amount) OVER (ORDER BY order_date)
```

Month on month change

Compare each row with the previous one.

```
revenue - LAG(revenue) OVER (ORDER BY
month)
```

Remove duplicates

Keep only the latest row per email.

```
ROW_NUMBER() OVER (PARTITION BY email
  ORDER BY created_at DESC)  -- keep rn =
1
```

@thedataguy16

Patterns 5 to 8

Joins and conditional logic. These separate a fresher answer from an experienced one.

Consecutive days

Users who logged in 3 days in a row. Same streak = same group value.

```
login_date - ROW_NUMBER() OVER (
  PARTITION BY user_id ORDER BY
login_date)
```

Find what is missing

Customers who never ordered.

```
LEFT JOIN orders o ON o.cust_id = c.id
WHERE o.id IS NULL
```

Pivot without PIVOT

Turn status rows into columns.

```
SUM(CASE WHEN status='Returned' THEN 1
ELSE 0 END)
```

Self join

Employees who earn more than their manager.

```
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary
```

@thedataguy16

How a window function works

Rows are not collapsed like GROUP BY. Each row keeps its detail and gets a new column.

dept	name	salary	rank
Sales	Asha	95	**1**
Sales	Ravi	95	**1**
Sales	Neha	80	**2**
Tech	Kiran	120	**1**
Tech	Meera	110	**2**
Tech	Arjun	90	**3**

Rank restarts at every new dept (PARTITION BY)

```
SELECT dept, name, salary,
  DENSE_RANK() OVER (PARTITION BY dept
                     ORDER BY salary
DESC) AS rank
FROM employees;
```

PARTITION BY

Splits rows into groups. Ranking restarts in each.

ORDER BY

Decides the order inside each group.

@thedataguy16

Your JOIN doubled the rows

Most asked debugging question. The answer is almost always duplicate keys.

```
-- Step 1: find the duplicate keys
SELECT customer_id, COUNT(*)
FROM orders
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Why it happens

1 row on the left matches 3 rows on the right, so you get 3 rows back. Sums get inflated.

Fix 1: aggregate first

GROUP BY on the right table, then join.

Fix 2: deduplicate

Use ROW_NUMBER() and keep rn = 1 before joining.

@thedataguy16

Order of execution

SQL does not run top to bottom. Knowing this answers 5 common questions at once.

```
1 FROM / JOIN      5 SELECT
2 WHERE            6 DISTINCT
3 GROUP BY         7 ORDER BY
4 HAVING           8 LIMIT
```

Alias fails in WHERE

SELECT runs after WHERE, so the alias does not exist yet.

WHERE vs HAVING

WHERE filters rows before grouping. HAVING filters groups after.

Alias works in ORDER BY

ORDER BY runs after SELECT.

@thedataguy16

CTE: write SQL people can read

Interviewers notice clean code. A CTE turns a nested mess into steps.

```
WITH monthly AS (
  SELECT DATE_TRUNC('month', order_date)
AS m,
         SUM(amount) AS revenue
  FROM orders GROUP BY 1
),
growth AS (
  SELECT m, revenue,
         LAG(revenue) OVER (ORDER BY m)
AS prev
  FROM monthly
)
SELECT m, revenue,
       ROUND(100.0*(revenue-prev)/prev,
1) AS mom_pct
FROM growth;
```

- **Readable**

  Each step has a name. Easy to debug one step at a time.

- **Reusable**

  Refer to the same CTE more than once in the query.

@thedataguy16

16 practice questions

Solve each using the 8 patterns. If you can do 12, you are interview ready.

1. Second highest salary without LIMIT
2. Top 3 products by revenue per category
3. Customers who ordered in Jan but not Feb
4. 7-day rolling average of sales
5. Users with 3+ consecutive login days
6. Remove duplicate emails, keep latest
7. % of total revenue by region
8. Employees earning more than their manager
9. First order date of every customer
10. Month on month growth %
11. COUNT(*) vs COUNT(col) vs COUNT(DISTINCT)
12. Why NULL = NULL is not true
13. Median salary per department
14. Month 2 retention of month 1 users
15. Highest order per customer with order details
16. How you would speed up a slow query

@thedataguy16